

Prova finale (Progetto di Reti Logiche)

Riccardo Marino Rocco [10515227]
Andrea Riza Zhabjaku [10575731]

1 Introduzione

Il progetto consiste nel realizzare un componente HW in VHDL. Tale componente data un'immagine in bianco e nero codificata su 8bit e di dimensione massima 128x128 deve essere in grado ricalibrarne il contrasto utilizzando una versione semplificata dell'algoritmo standard per l'equalizzazione di un istogramma.

L'interfaccia del componente è definita come da specifica:

```
-- Port definition by project specification--
entity project_reti_logiche is
  port (
    i_clk      : in std_logic;
    i_rst      : in std_logic;
    i_start    : in std_logic;
    i_data     : in std_logic_vector(7 downto 0);
    o_address  : out std_logic_vector(15 downto 0);
    o_done     : out std_logic;
    o_en       : out std_logic;
    o_we       : out std_logic;
    o_data     : out std_logic_vector (7 downto 0);
  );
end project_reti_logiche;
```

```
DELTA_VALUE = MAX_PIXEL_VALUE - MIN_PIXEL_VALUE
SHIFT_LEVEL = (8 - FLOOR(LOG2(DELTA_VALUE + 1)))
TEMP_PIXEL = (CURRENT_PIXEL_VALUE - MIN_PIXEL_VALUE) << SHIFT_LEVEL
NEW_PIXEL_VALUE = MIN( 255 , TEMP_PIXEL)
```

Versione semplificata dell'algoritmo da implementare

2 Descrizione dei segnali

Il modulo comunica con una **RAM** nella quale da specifica sono precaricati i byte dell'immagine accodati a 2 byte contenenti la dimensione dell'immagine espressa in **COLONNExRIGHE**.

La computazione viene avviata dal segnale **i_start** e gestita dal test tramite i segnali **i_clk** (segnale di clock operativo del componente) e **i_rst** (segnale di reset della computazione, inizializza il componente).

La comunicazione con la memoria **RAM** viene abilitata tramite il segnale **o_en** (alto per le operazioni sia di lettura che di scrittura). Per l'operazione di scrittura è richiesto che sia alto anche il segnale **o_we** di **WRITE ENABLE**.

i_data e **o_data** sono i segnali che leggono/scrivono, da e verso la memoria, la parola all'indirizzo **o_address**.

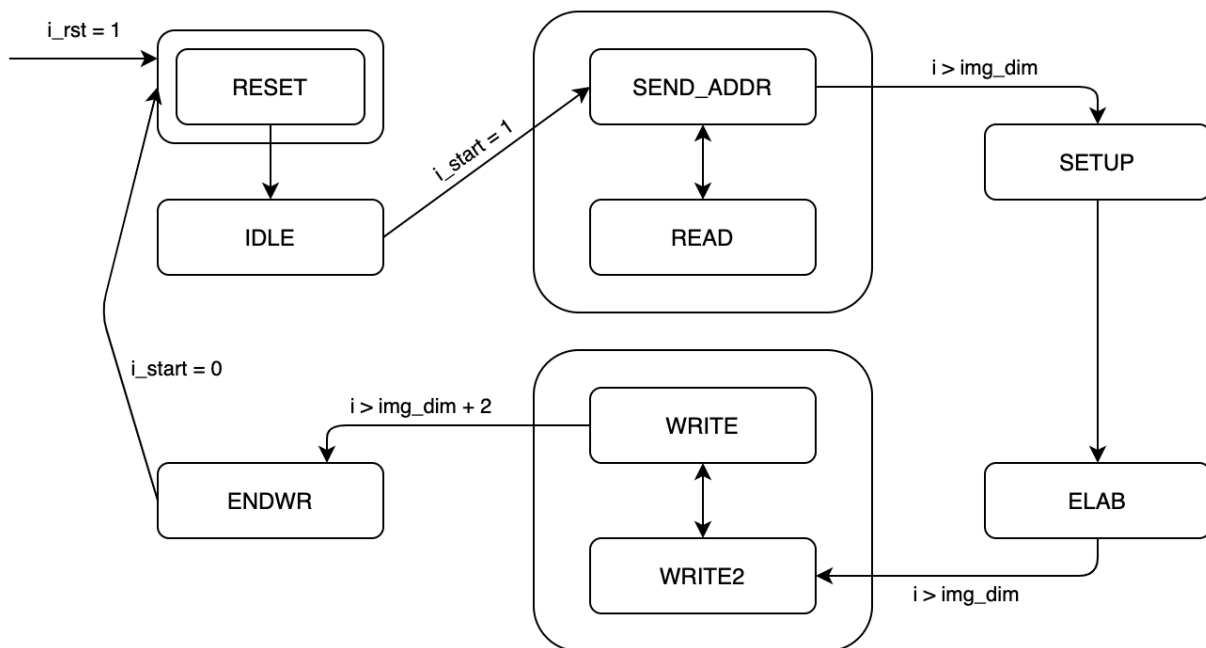
Il termine della computazione viene indicato dal segnale **o_done**, alto solo dopo aver scritto il risultato in memoria.

Il componente è in grado di effettuare un reset completo e mettersi in attesa di un nuovo segnale di start ogni volta che **i_rst=1**, indipendentemente dallo stato in cui si trova.

3 Architettura

Come suggerisce la specifica, la soluzione più ovvia è quella di usare una macchina a stati finiti per i vari step dell'elaborazione. Analizzando la complessità del problema abbiamo optato per una soluzione basata su un singolo process (monomodulare) che scandisce i vari stati ed esegue le operazioni elementari nel contesto della soluzione.

3.1 FSM



3.2 Gli stati

In ogni stato vengono preparati le uscite dei segnali che interagiscono con la RAM in modo tale da avere nel clock successivo i valori utili per la computazione dello stato successivo.

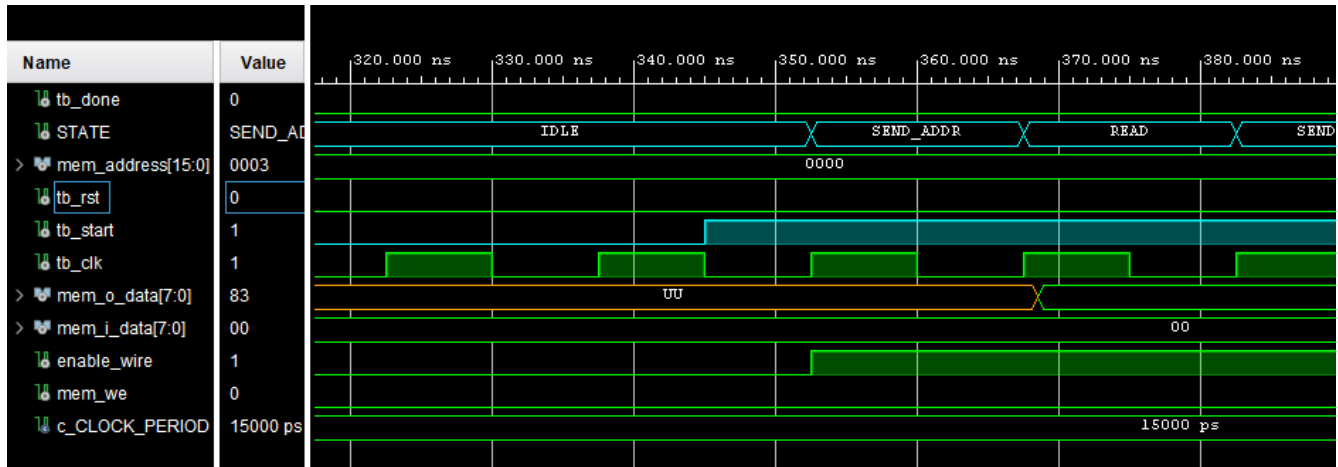
3.2.1 RESET

È lo stato iniziale del FSM in cui vengono inizializzati i registri.

Lo stato presenta due condizioni di entrata: il segnale $i_rst=1$ o il segnale $i_start=0$ (passando obbligatoriamente da **ENDWR** quindi solo dopo aver posto $o_done=1$ e l'elaborazione dell'immagine precedente è stata completata con successo).

3.2.2 IDLE

È lo stato di attesa del FSM con condizione di uscita: segnale **i_start=1** (comando di inizio dell'elaborazione di una nuova immagine).



Transizione dell'automa nello stato di SEND_ADDR a seguito di un segnale start=1

3.2.3 SEND_ADDR

Uno stato necessario per poter effettuare nello stato di **READ** la corretta lettura del dato ricevuto in ingresso che altrimenti leggerebbe il dato della richiesta precedente. Gioca quindi un ruolo importante per la corretta temporizzazione del componente.

È qui che avviene il passaggio nello stato successivo a lettura completata; infatti quando l'indirizzo da richiedere è superiore alla dimensione dell'immagine **img_dim** abbiamo il passaggio allo stato di **SETUP** dopo aver calcolato **delta_value** (la differenza tra **max_pixel_value** e **min_pixel_value**).

3.2.4 READ

È lo stato in cui il componente legge il dato in ingresso dalla **RAM** e lo salva nella sua memoria. Il salvataggio del dato letto è necessario data la necessità di avere una lettura e scrittura sequenziale in **RAM**.

In questo stato calcoliamo anche **img_dim** (la dimensione dell'immagine data dal prodotto del contenuto dei primi due indirizzi della **RAM**) e i valori **max_pixel_value** e **min_pixel_value** richiesti per l'applicazione dell'algoritmo.

Qui viene anche effettuato un controllo sui primi due byte in memoria relativi alla dimensione dell'immagine. Si conclude direttamente la computazione nel caso in cui una dimensione (o entrambe) sia pari a zero.

3.2.5 SETUP

Stato raggiungibile da **SEND_ADDR** nel momento in cui, nota la dimensione dell'immagine, il componente termina la lettura di quest'ultima dalla memoria. È lo stato in cui si calcola lo **shift_level** da applicare al pixel (operazione effettuata poi nello stato di **ELAB**).

Lo **shift_level** viene individuato effettuando il calcolo di un logaritmo in base 2 arrotondato all'intero precedente. Il calcolo viene effettuato tramite la ricerca del bit più significativo di **delta_value**.

Per questioni di temporizzazione in questo stato avviene il calcolo del **temp_pixel** a meno dello **shift_level**.

3.2.6 ELAB

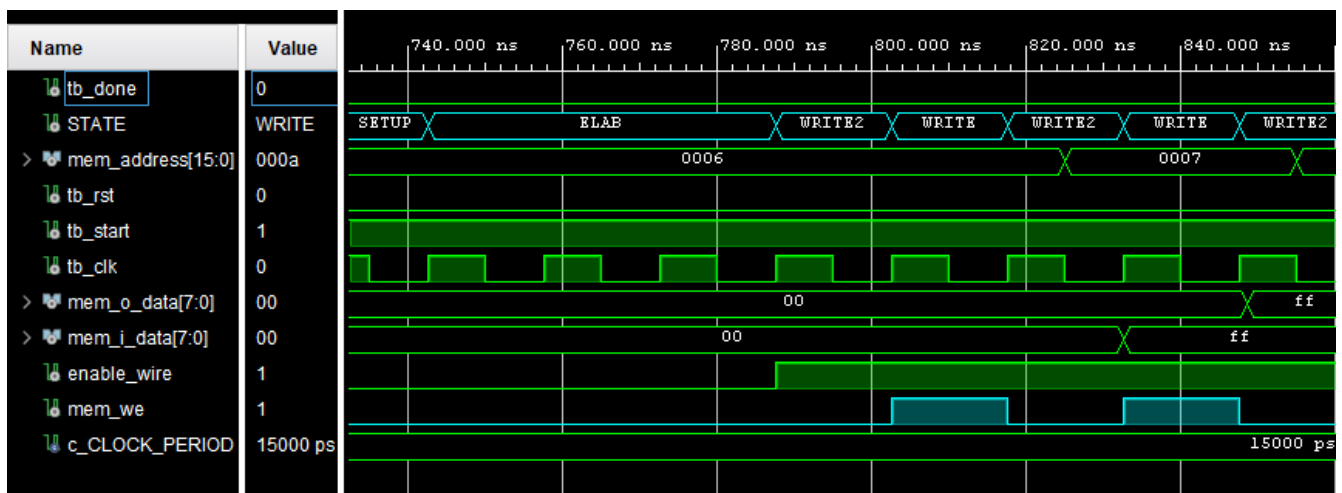
In questo stato viene portato a termine il calcolo del **temp_pixel** applicando lo **shift_level** calcolato nello stato di **SETUP**.

Alla fine di ogni elaborazione si torna nello stato di **SETUP** per potersi preparare all'elaborazione del pixel successivo.

Quando la lettura di tutti i pixel dell'immagine è terminata l'FSM è in grado di transizionare nello stato di **WRITE**.

3.2.7 WRITE

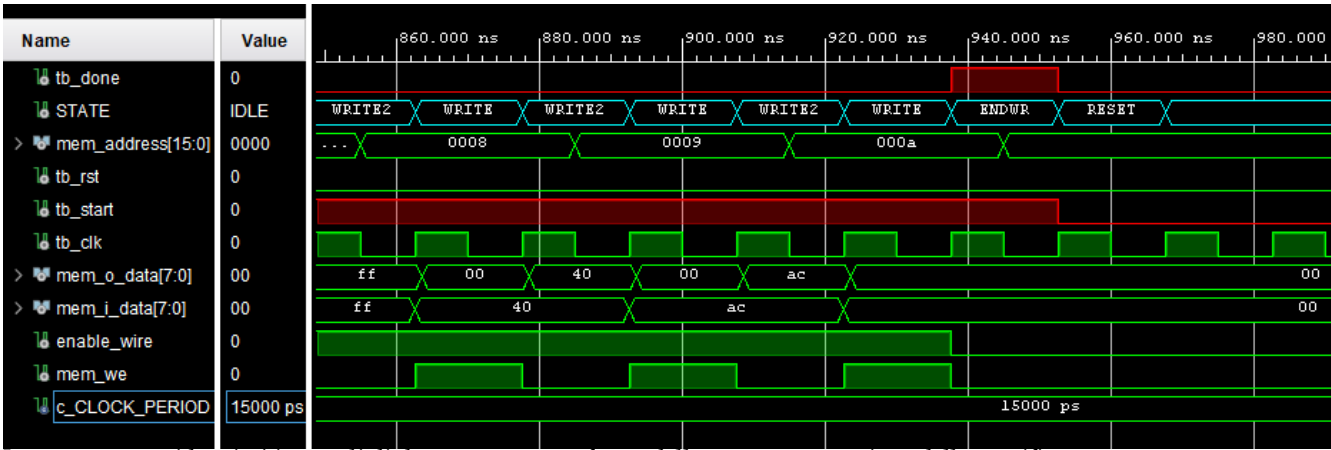
Lo stato in cui il componente invia sequenzialmente alla **RAM** i nuovi pixel dell'immagine equalizzata aspettando il segnale di write enable **o_we**, come si può osservare nella waveform qui allegata:



Per facilitare la comprensione in blu i segnali di write enable e gli stati del FSM

3.2.8 ENDWR

Stato di transizione tra **WRITE** e **RESET** in quanto da specifica è richiesto che il segnale **i_start** debba essere prima riportato a **zero** (a meno di un **i_rst=1**). Per tutto il tempo in cui lo stato di **ENDWR** è posto il segnale di **o_done=1** e verrà posto a zero solamente al passaggio nello stato di **RESET**, come da specifica.



In rosso sono evidenziati i segnali di done e start a conferma della corretta esecuzione della specifica

4 Risultati sperimentali

4.1 Sintesi

Il componente è sintetizzabile e implementabile con un totale di 3942 LUT e 8217 FF. Il risultato è stato però ottenuto riducendo la grandezza della memoria del componente in quanto, probabilmente per limiti delle nostre macchine, una memoria di 17.000 byte portava ad un arresto anomalo di Vivado durante la sintesi.

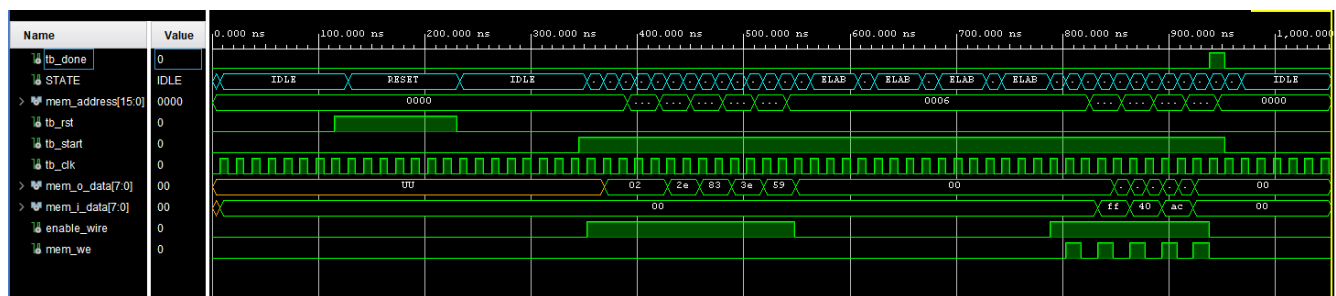
The screenshot displays the Vivado IDE interface. On the left, the 'Properties' window for the project 'project_reti_logiche' is visible, showing details like Location, Type (VHDL), Library (xil_defaultlib), Size (7.9 KB), and Modified date. The main window shows the 'Design Runs' tab with a table of synthesis results. The table includes columns for Name, Constraints, Status, WNS, TNS, WHS, THS, TPWS, Total Power, Failed Routes, LUT, FF, BRAM, URAM, DSP, and Start. The 'synth_1' run is marked as 'Complete' with 3942 LUTs and 8217 FFs. The 'impl_1' run is 'Queued...'. On the right, a snippet of VHDL code is shown, defining the architecture 'Behavioral of project_reti_logiche' with various signals and types.

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAM	URAM	DSP	Start
✓ synth_1	constrs_1	synth_design Complete!								3942	8217	0.0	0	0	9/12/21, 2
⚙ impl_1	constrs_1	Queued...													

Esito della sintesi con dimensione di MEM ridotta a 1000 byte

4.2 Simulazioni

4.2.1 TestBench immagine 2x2 (fornito dal docente)



Test di un'immagine 2x2. In blu gli stati del FSM

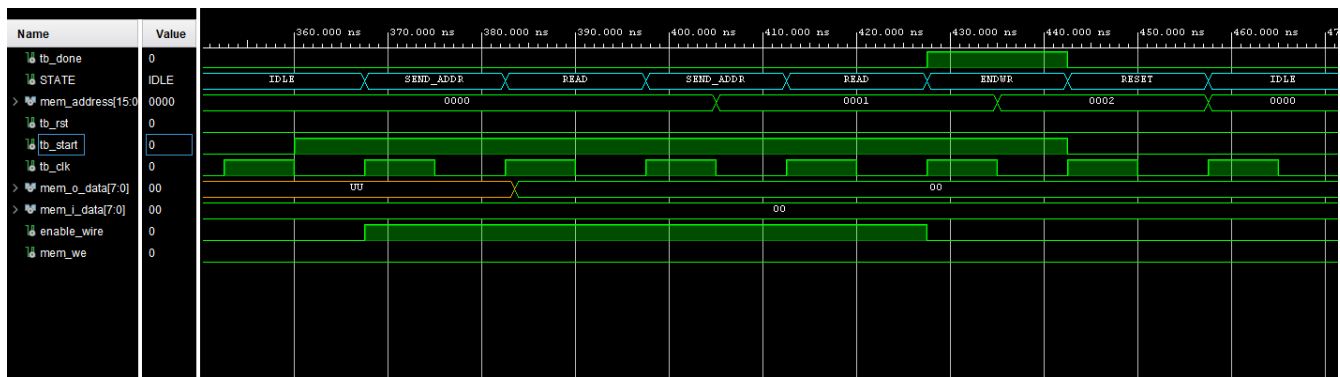
4.2.2 Test di dimensione massima 128x128

Il test viene eseguito completamente e correttamente in 1,5ms.

Non alleghiamo istantanee della waveform in quanto per motivi di spazio legati alla dimensione del test non sarebbe di facile comprensione e risulterebbe quindi poco utile ai fini della relazione.

4.2.3 Test caso limite: dimensione 0x0

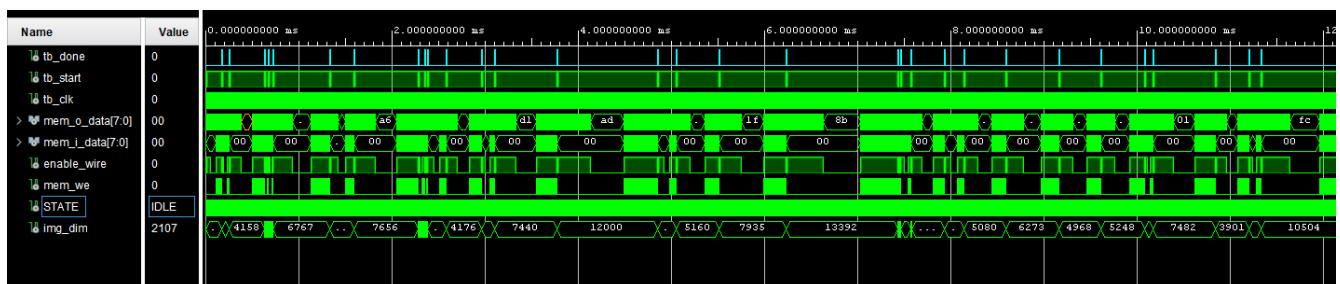
Possiamo notare come nel momento in cui l'immagine è di dimensione nulla il controllo effettuato nello stato di **READ** permette al componente di terminare la computazione transizionando nello stato di **ENDWR**:



Test singolo di un'immagine vuota, contenuto della RAM=[0,0]

4.2.4 Ulteriori test

Sono state inoltre eseguite simulazioni dell'equalizzazione di più immagini usato set di 100, 500, 2.000 e 10.000 test generati randomicamente di dimensioni comprese tra 0 e 128 pixel (righe e colonne).



Esempio di esecuzione di 30 test consecutivi, facilmente identificabili dai segnali di done=1

Nota: I test sono stati generati utilizzando un generatore messo a disposizione da un compagno di corso. Il componente riusciva a passare perfettamente i test generati utilizzando la testbench fornita insieme al generatore.

E' stato poi successivamente effettuato un refactoring di generatore e testbench e improvvisamente non venivano più riconosciuti come corretti gli output del componente. Dopo numerose verifiche siamo arrivati alla conclusione che la testbench potesse essere sbagliata e infatti grazie al consiglio del professore ci siamo accorti che veniva eseguito il controllo dell'output confrontandolo col pixel precedente e non quello nella posizione corretta in memoria. Risolto il problema possiamo affermare che l'output del componente è corretto.